| Ex No : 7<br>Date:01/10/26 | GENERATIVE MODELS WITH GANS |
|---|---|

**AIM:**

To construct and train a **Generative Adversarial Network (GAN)** using **TensorFlow/Keras** on the **MNIST dataset** to generate new synthetic images of handwritten digits that resemble the original MNIST images.

**ALGORITHM:**

**Generative Adversarial Network (GAN)**

1. Load the MNIST Dataset ○ Load handwritten digit images from the MNIST dataset.
   - Reshape the images into 28 × 28 × 1.
   - Normalize pixel values to the range [-1, 1].
2. Build the Generator ○ Take a random noise vector of size 100 as input. ○ Use Dense, Batch Normalization, LeakyReLU and Conv2DTranspose layers.
   - Generate a synthetic 28 × 28 × 1 image.
   - Use tanh activation in the output layer.
3. Build the Discriminator ○ Take a 28 × 28 × 1 image as input.
   - Use convolutional layers to extract features.
   - Use Flatten and Dense layers.
   - Output 1 for a real image and 0 for a fake image.
4. Define Loss Functions ○ Use Binary Cross-Entropy loss.
   - Discriminator learns to identify real and fake images.
   - Generator learns to fool the discriminator.
5. Train the GAN ○ Generate fake images using random noise. ○ Train the discriminator using real and fake images. ○ Train the generator using the discriminator's output.
   - Update both networks using the Adam optimizer.

6. Repeat the Training ○ Repeat the process for multiple epochs. ○ As training progresses, the generator produces increasingly realistic handwritten digits.
7. Generate Images ○ Use a fixed random seed to generate images. ○ Rescale the generated images from [-1,1] to [0,1]. ○ Save and display the generated images.

**CODE:**

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.datasets import mnist
import os

tf.keras.utils.disable_interactive_logging()

(x_train, _), (_, _) = mnist.load_data()

x_train = x_train.reshape(x_train.shape[0], 28, 28, 1).astype("float32")
x_train = (x_train - 127.5) / 127.5

latent_dim = 100

def build_generator():
    model = keras.Sequential(name="generator")
    model.add(layers.Dense(7 * 7 * 256, use_bias=False, input_shape=(latent_dim,)))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())
    model.add(layers.Reshape((7, 7, 256)))

    model.add(layers.Conv2DTranspose(128, (5, 5),
                                     strides=(1, 1),
                                     padding="same",
                                     use_bias=False))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())

    model.add(layers.Conv2DTranspose(64, (5, 5),
                                     strides=(2, 2),
                                     padding="same",
                                     use_bias=False))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())

    model.add(layers.Conv2DTranspose(1, (5, 5),
                                     strides=(2, 2),
                                     padding="same",
```

```
                                    use_bias=False,
                                    activation="tanh"))
    return model

generator = build_generator()

def build_discriminator():
    model = keras.Sequential(name="discriminator")

    model.add(layers.Conv2D(64, (5, 5),
                            strides=(2, 2),
                            padding="same",
                            input_shape=[28, 28, 1]))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))

    model.add(layers.Conv2D(128, (5, 5),
                            strides=(2, 2),
                            padding="same"))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))

    model.add(layers.Flatten())
    model.add(layers.Dense(1, activation="sigmoid"))

    return model

discriminator = build_discriminator()

cross_entropy = keras.losses.BinaryCrossentropy(from_logits=False)

def discriminator_loss(real_output, fake_output):
    real_loss = cross_entropy(tf.ones_like(real_output), real_output)
    fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
    return real_loss + fake_loss

def generator_loss(fake_output):
    return cross_entropy(tf.ones_like(fake_output), fake_output)

generator_optimizer = tf.keras.optimizers.Adam(learning_rate=1e-4)
discriminator_optimizer = tf.keras.optimizers.Adam(learning_rate=1e-4)

EPOCHS = 200
batch_size = 256
```

```
EPOCHS = 200
batch_size = 256
num_examples_to_generate = 16

@tf.function
def train_step(images):
    noise = tf.random.normal([batch_size, latent_dim])

    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:

        generated_images = generator(noise, training=True)

        real_output = discriminator(images, training=True)
        fake_output = discriminator(generated_images, training=True)

        gen_loss = generator_loss(fake_output)
        disc_loss = discriminator_loss(real_output, fake_output)

    gradients_of_generator = gen_tape.gradient(
        gen_loss,
        generator.trainable_variables
    )

    gradients_of_discriminator = disc_tape.gradient(
        disc_loss,
        discriminator.trainable_variables
    )

    generator_optimizer.apply_gradients(
        zip(gradients_of_generator, generator.trainable_variables)
    )

    discriminator_optimizer.apply_gradients(
        zip(gradients_of_discriminator, discriminator.trainable_variables)
    )

    return gen_loss, disc_loss

def generate_and_save_images(model, epoch, test_input):
    predictions = model(test_input, training=False)
```

```
def generate_and_save_images(model, epoch, test_input):
    predictions = model(test_input, training=False)

    predictions_rescaled = (predictions * 0.5) + 0.5

    fig = plt.figure(figsize=(4, 4))

    for i in range(predictions.shape[0]):
        plt.subplot(4, 4, i + 1)
        plt.imshow(predictions_rescaled[i, :, :, 0], cmap="gray")
        plt.axis("off")

    plt.suptitle(f"Epoch {epoch}", fontsize=16)

    if not os.path.exists("images"):
        os.makedirs("images")

    plt.savefig(f"images/image_at_epoch_{epoch:04d}.png")
    plt.show()

seed = tf.random.normal(
    [num_examples_to_generate, latent_dim]
)

train_dataset = (
    tf.data.Dataset
    .from_tensor_slices(x_train)
    .shuffle(x_train.shape[0])
    .batch(batch_size)
)

def train(dataset, epochs):

    for epoch in range(epochs):

        gen_loss_list = []
        disc_loss_list = []

        for image_batch in dataset:

            gen_loss, disc_loss = train_step(image_batch)
```

OUTPUT:

```
--- Part 1: K-Means Clustering ---

Original K-Means Dataset Head:
   Feature_1  Feature_2
0  -7.155244  -7.390016
1  -7.395875  -7.110843
2  -2.015671   8.281780
3   4.509270   2.632436
4  -8.102502  -7.484961
```

```
--- Part 1: Loading and Preprocessing the MNIST Dataset ---
Normalized training data shape: (60000, 28, 28, 1)
Example normalized pixel value: -1.0
```

```
--- Beginning GAN Training ---
Epoch 1/20 - Generator Loss: 0.7877, Discriminator Loss: 1.0228
Epoch 2/20 - Generator Loss: 0.8148, Discriminator Loss: 1.2225
Epoch 3/20 - Generator Loss: 0.8448, Discriminator Loss: 1.3034
Epoch 4/20 - Generator Loss: 0.8534, Discriminator Loss: 1.2366
Epoch 5/20 - Generator Loss: 0.8372, Discriminator Loss: 1.2497
```

Epoch 5



```
Epoch 6/20 - Generator Loss: 0.8516, Discriminator Loss: 1.2705
Epoch 7/20 - Generator Loss: 0.8888, Discriminator Loss: 1.3028
Epoch 8/20 - Generator Loss: 0.8739, Discriminator Loss: 1.2512
Epoch 9/20 - Generator Loss: 0.8691, Discriminator Loss: 1.3130
Epoch 10/20 - Generator Loss: 0.8862, Discriminator Loss: 1.2320
```

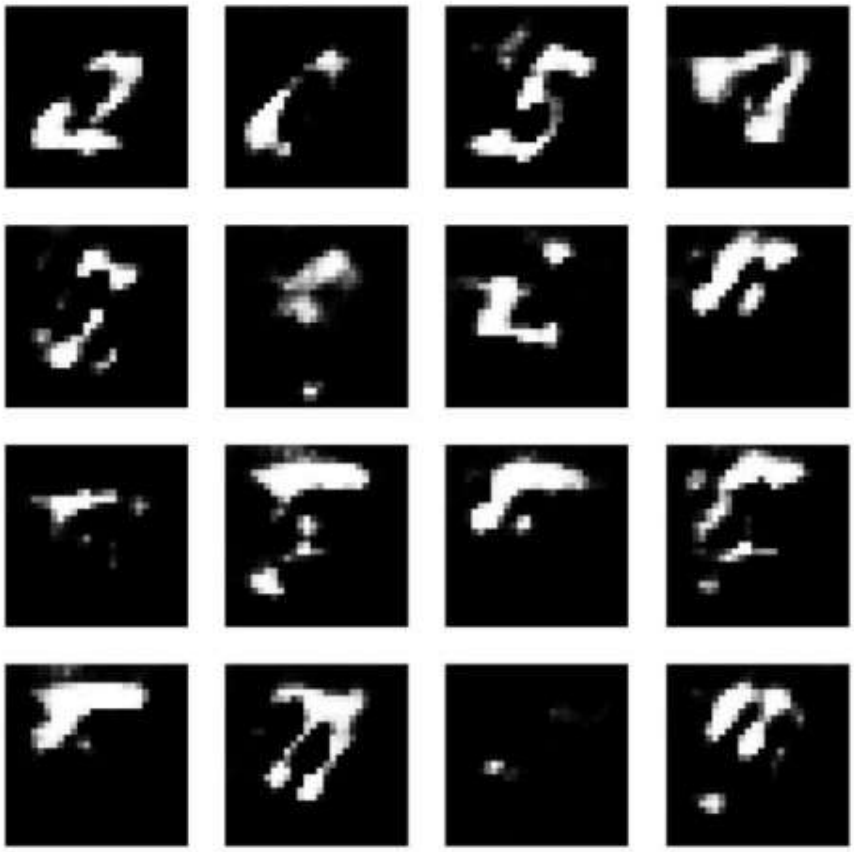
Epoch 10





```
Epoch 11/20 - Generator Loss: 0.9361, Discriminator Loss: 1.2244
Epoch 12/20 - Generator Loss: 0.9946, Discriminator Loss: 1.1719
Epoch 13/20 - Generator Loss: 0.9948, Discriminator Loss: 1.1944
Epoch 14/20 - Generator Loss: 0.9786, Discriminator Loss: 1.1809
Epoch 15/20 - Generator Loss: 1.0420, Discriminator Loss: 1.1079
```

Epoch 15



```
Epoch 16/20 - Generator Loss: 1.2020, Discriminator Loss: 1.0483
Epoch 17/20 - Generator Loss: 1.2648, Discriminator Loss: 1.0605
Epoch 18/20 - Generator Loss: 1.1657, Discriminator Loss: 1.0404
Epoch 19/20 - Generator Loss: 1.1644, Discriminator Loss: 1.0897
Epoch 20/20 - Generator Loss: 1.1770, Discriminator Loss: 1.0938
```

Epoch 20



--- Training complete. Generating final images. ---




**RESULT:**

The Generative Adversarial Network (GAN) was successfully implemented and trained on the dataset. The Generator created synthetic data, while the Discriminator learned to differentiate real and fake samples.

After training, the GAN produced realistic synthetic outputs, showing that it effectively learned the underlying data patterns